EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

Week 12: OPERATION PALE HORSE

ARM Cortex-M33 • GDB • Ghidra

November 14, 2026

For educational and defensive security research only

Reverse Engineering a Three-Layer Locked RP2350 Firmware with GDB, OpenOCD, and Ghidra

LEGAL DISCLAIMER: The information, tools, and code provided in this repository and course are strictly for educational, research, and defensive purposes only.

You are explicitly prohibited from using any materials contained herein to access, test, modify, or exploit any device, network, or system that you do not own 100% or for which you do not have explicit, documented, and legally binding authorization to interact with.

By using this repository and course, you acknowledge and agree that:

1. Any illegal, unauthorized, or malicious use of this information is solely your responsibility.
2. The author(s) and contributor(s) of this repository and course shall not be held liable for any damages, legal repercussions, criminal charges, or unauthorized actions resulting from the use, misuse, or abuse of the contents herein.
3. You will comply with all applicable local, state, national, and international laws regarding cybersecurity and computer fraud.

IF YOU DO NOT AGREE WITH THESE TERMS, DO NOT USE THIS REPOSITORY AND COURSE.

What You'll Learn This Week

By the end of this week, you will be able to:

- Explain what a firmware image is, and the difference between a `.bin` and a `.elf`
- Explain the RP2350 vector table and why the first two words of a binary are not code
- Connect a Debug Probe over SWD, start OpenOCD, and attach GDB to a running RP2350
- Set hardware breakpoints correctly on code that lives in flash, and explain why software breakpoints fail there
- Read a raw `.bin` in Ghidra by hand: choose the processor, set the base address, and force Thumb mode
- Reverse a CRC-32 integrity gate and recompute it after a patch
- Reverse an XOR keystream and a self-modifying first word
- Watch a program decrypt itself into RAM and jump into that RAM, instruction by instruction
- Find an off-by-one buffer overflow and prove it by driving the exact store instruction under GDB
- Reverse a challenge and response routine and compute a valid answer for any challenge
- Find a hidden command that is present in the dispatch table but absent from the help text
- Explain why the RP2350 cannot be permanently bricked in this lab

This week is the capstone. It uses every skill from Weeks 1 through 11 at once.

Before You Begin: Hardware and Software

You need the following. Check each box before you start.

Item	What it is
Raspberry Pi Pico 2	The RP2350 board that runs the firmware
Raspberry Pi Debug Probe	The device that connects your computer to the board's CPU over SWD
USB cable for the Debug Probe	Connects the probe to your computer
USB cable for the Pico 2	Powers the board and exposes BOOTSEL
USB to UART adapter (optional but recommended)	Lets you see the serial console on GP0 and GP1
Three jumper wires	SWDIO, SWCLK, and GND between probe and board
Arm GNU Toolchain (GDB and objdump)	The debugger and binary tools
OpenOCD	The program that speaks SWD and serves GDB
Ghidra	The static analysis and decompiler tool

Unless a step says otherwise, run shell commands from the repository root and use the paths exactly as shown.

The commands in this document were captured on macOS with:

- Open On-Chip Debugger 0.12.0+dev-gcf9c0b41c (2025-02-16-15:07)
- GNU gdb (Arm GNU Toolchain 14.2.Rel1, build arm-14.52) 15.2.90
- Ghidra 12.1.4

On Windows and Linux the paths differ but the commands are the same.

Part 1: The Mission

A box arrived at your desk. It is a Raspberry Pi Pico 2 running firmware that belongs to a company called Fun Society Amusement, LLC. The company does not exist. The board is a dead drop terminal: a machine built to hold a message and to keep people out.

The firmware was written to stop you in three ways.

Lock 1. The integrity gate. Before the board runs its main program, it computes a fingerprint of that program and compares it to a stored copy. Change one byte and the board refuses to start.

Lock 2. The hidden room. The most important code is stored scrambled. At startup the loader unscrambles it into RAM, patches one word, and jumps into RAM to run it. If you look only at flash, you see noise.

Lock 3. The console. The board takes commands over a serial cable. One command is hidden from the menu. The line reader has an off-by-one bug. And a separate module, the ghost, implements a challenge and response that changes every time.

Your job is to understand every one of these, prove each with real evidence from the board, and explain them to someone else. That is the capstone.

The rest of this document is how to do it. It does not skip steps, and every command and every line of real output below was produced by the actual board and the actual tools.

Part 2: Foundations You Must Know

Read this part even if you think you know it. Every later step depends on these ideas.

2.1 A program is a list of numbers

The CPU does not read English. It reads numbers. Each number is an **instruction**. On the RP2350's ARM core, for example, the two bytes `70 47` are an instruction that means "return from this function".

A program is a long row of these numbers in memory. The CPU reads the number at the address held in the **program counter** (`pc`), does what it says, moves `pc` forward, and repeats. That is all a computer does, billions of times per second.

2.2 The `.bin` file is the program. The `.elf` adds a map

The `.bin` file is a byte for byte copy of the numbers that are written into flash. Nothing is labeled. If you open it, you are looking at the program itself.

The `.elf` file is the same numbers plus two things:

- names for functions and variables,
- a map of which bytes go where in memory.

The `.elf` is convenient if you built the program. It is not what an investigator usually gets. For most of this week you work on the `.bin` , exactly like a real reverse engineer.

The build directory contains both. We use the `.bin` to investigate and the `.elf` only to check our answers.

2.3 The first two words are not code

The first eight bytes of the image are two 32 bit numbers:

- **word 1:** the address of the top of the stack,
- **word 2:** the address of the first instruction.

The CPU reads these two numbers the moment it leaves reset. That is how it knows where to begin. These two words are called the **vector table**.

On the shipped image the real values are:

```
0x10000000:  0x20082000      0x10000089
```

`0x20082000` is the stack top. `0x10000089` is the start address. The trailing `9` is not a typo. On ARM Cortex-M, an odd address means "the code at this address is Thumb code". Clear the low bit and you get `0x10000088` , the real first instruction.

2.4 Flash is permanent, RAM is a scratchpad, and XIP means run from flash

Flash keeps its contents when power is removed. The program lives there, starting at `0x10000000` .

RAM forgets everything when power is removed. The board uses it for the stack, variables, and anything it builds at runtime, around `0x20000000` .

The RP2350 can run code directly from flash. This is called **XIP**, short for Execute In Place. It means the CPU does not have to copy flash code into RAM to run it.

But it can also run code from RAM. That is central to this week, because the ghost is unscrambled into RAM and executed there.

2.5 Registers are the CPU's hands

The CPU cannot do arithmetic on memory directly. It loads numbers into small fast boxes called **registers**, works on them, and stores the results back.

Register	Name	What it holds
`r0` to `r12`	general purpose	arguments, results, and working values
`r13` or `sp`	stack pointer	top of the stack
`r14` or `lr`	link register	return address after a function call
`r15` or `pc`	program counter	address of the next instruction

When a function is called, the caller puts arguments in `r0`, `r1`, `r2`, `r3`. The function puts its answer in `r0`.

2.6 The three locks at a glance

Lock	Where it lives	What it does	How to beat it
1. Integrity gate	`0x10000008`	CRC-32 of the app compared to `0x1003f000`	recompute and rewrite the CRC
2. Hidden room	`0x10000054`	XOR unscramble into `0x20040000`, then jump there	read the key and unscramble it yourself
3. Console	`0x10001090`	hidden command plus off-by-one in the reader	find the table entry and the bug

Part 3: The Firmware You Are Given

3.1 The address map

The image is one 256 KB file. It has four meaningful regions.

Address in memory	What it is	Notes
`0x10000000`	Loader	vectors at the very start, then loader code
`0x10001000`	Main program	vectors, then the console
`0x1003e000`	Scrambled ghost	4096 bytes of noise in flash
`0x1003f000`	Manifest	stored CRC-32 and the company tag

The loader is small. The main program is padded to `0x4000` bytes so its size is fixed. That fixed size matters because it is the length the CRC covers.

3.2 The shipped binary

Use the binary that ships with the lab. Do not rebuild it. The RP2350 SDK changes over time, and a newer version can produce different bytes or fail to build this project.

```
shasum -a 256 WEEK12/0x002a_pale-horse/build/pale-horse.bin
```

Real output:

```
3947e3f526283a355570b0a4684bd2a459d9bbcf61ca303bf821e24c84679b72  WEEK12/0x002a_pale-
horse/build/pale-horse.bin
```

And its size:

```
stat -f "%z bytes" WEEK12/0x002a_pale-horse/build/pale-horse.bin
```

Real output on macOS:

```
262144 bytes
```

If your SHA-256 does not match, you are not looking at the lab image. Stop and get the shipped file.

3.3 What is inside

`strings` finds the readable text without running anything. We lower the minimum length to three so short command names like `led` are not skipped. Real output from the shipped image:

```
strings -n 3 WEEK12/0x002a_pale-horse/build/pale-horse.bin | grep -E "FUN SOCIETY|DEAD DROP|AUTH
REQUIRED|help status|stage3|\[led\]|\[lock\]|ASSET LEDGER|^help$|^status$|^led$|^unlock$|^factory$"
```

Real output:

```
FUN SOCIETY AMUSEMENT, LLC
DEAD DROP TERMINAL v0.9
AUTH REQUIRED
help status led unlock
[stage3] running
[led] toggled
[lock] challenge issued
[factory] FUN SOCIETY AMUSEMENT, LLC
ASSET LEDGER PALE-HORSE-7F3A
help
status
led
unlock
factory
QFUN SOCIETY AMUSEMENT, LLC:
```

The last line comes from the manifest. The bytes before the tag are the CRC, and the low byte of the CRC happens to be the printable character `Q`, so `strings` starts the match at that byte.

Notice several things already.

- The help text lists `help status led unlock`. It does not list `factory`.

- But the word `factory` is in the binary, and there is a `[factory]` message.
- There is an asset ledger line: `ASSET LEDGER PALE-HORSE-7F3A`.

That mismatch is your first real clue, and you found it before using any debugger.

3.4 The very first bytes

```
xxd -g 4 -l 8 WEEK12/0x002a_pale-horse/build/pale-horse.bin
```

Real output:

```
00000000: 00200820 89000010                    . . ....
```

Read as little endian, that is `0x20082000` and `0x10000089`: the stack top and the start address. That is the vector table from Part 2.

Part 4: Preparing the Board and the Tools

4.1 Install the Arm GNU Toolchain

You need `arm-none-eabi-gdb` and `arm-none-eabi-objdump`.

On macOS the toolchain installs to a versioned directory. Our captured session used:

```
/Applications/ArmGNUToolchain/14.2.rel1/arm-none-eabi/bin/arm-none-eabi-gdb
```

Check it:

```
arm-none-eabi-gdb --version
```

Real output:

```
GNU gdb (Arm GNU Toolchain 14.2.Rel1 (Build arm-14.52)) 15.2.90.20241130-git
```

Important: use the 14.2 release or later. In earlier testing, GDB 13.3 would not connect to this RP2350 target, while 14.2 connected immediately.

4.2 Install OpenOCD

OpenOCD is the program that talks SWD to the probe and serves GDB on a port. On macOS we used a local build:

```
~/openocd/src/openocd --version
```

Real output:

```
Open On-Chip Debugger 0.12.0+dev-gcf9c0b41c (2025-02-16-15:07)
```

The important part is that the `rp2350` target config exists under `~/openocd/tcl/target/rp2350.cfg`.

4.3 Install Ghidra

Ghidra needs a Java runtime. We used Ghidra 12.1.4 with OpenJDK 21.

```
ls /Applications/ghidra_12.1.4_PUBLIC/support/analyzeHeadless
```

Real output:

```
/Applications/ghidra_12.1.4_PUBLIC/support/analyzeHeadless
```

4.4 Wire the Debug Probe to the board

The probe talks to the board over SWD, a two wire debug bus. Make these three connections.

Probe pin	Pico 2 pin	Why
SWDIO	SWDIO (debug header)	data
SWCLK	SWCLK (debug header)	clock
GND	GND	common ground

Connect the probe's USB to your computer. Connect the Pico 2's USB to your computer for power. Do not connect the probe's power pin.

When the probe is connected correctly, OpenOCD will report a serial number. Ours was:

```
Info : Using CMSIS-DAPv2 interface with VID:PID=0x2e8a:0x000c, serial=E6647C740371992F
```

Your serial number will be different. What matters is that OpenOCD finds a CMSIS-DAP probe.

4.5 Wire the serial console (optional but recommended)

The firmware prints on **UART0**, on **GP0** (board transmit) and **GP1** (board receive), at **115200** bits per second.

This is not the USB cable. The USB port is used for power and for BOOTSEL flashing. To see the console you connect a USB to UART adapter:

Board pin	Adapter pin
GP0 (UART0 TX)	adapter RX
GP1 (UART0 RX)	adapter TX
GND	GND

Open the adapter's serial port at 115200, 8 data bits, no parity, 1 stop bit.

If you connect only the Pico 2 USB cable and open the USB serial port, you will see nothing. The firmware is not printing on USB. This trips up almost everyone the first time.

4.6 Flash the shipped binary

From the repository root, with the probe connected and no other debugger running:

```
~/openocd/src/openocd -s ~/openocd/tcl \
  -f interface/cmsis-dap.cfg \
  -f target/rp2350.cfg \
  -c "adapter speed 1000" \
  -c init \
  -c "program WEEK12/0x002a_pale-horse/build/pale-horse.bin 0x10000000 verify reset exit"
```

Real output, trimmed to the important lines:

```
Info : Using CMSIS-DAPv2 interface with VID:PID=0x2e8a:0x000c, serial=E6647C740371992F
Info : CMSIS-DAP: SWD supported
Info : clock speed 1000 kHz
Info : SWD DPIDR 0x4c013477
Info : [rp2350.dap.core0] Cortex-M33 r1p0 processor detected
Info : [rp2350.dap.core0] target has 8 breakpoints, 4 watchpoints
** Programming Started **
** Programming Finished **
** Verified OK **
** Verified OK **
** Resetting Target **
```

`Verified OK` means OpenOCD read the flash back and the bytes match the file. If you do not see it, the write failed; check the probe connection and lower the adapter speed.

4.7 Confirm it is running

With the USB to UART adapter on GP0 and GP1, open the port at 115200. You will see the exact banner that is stored at `0x10001620` in the image:

```
FUN SOCIETY AMUSEMENT, LLC
DEAD DROP TERMINAL v0.9
AUTH REQUIRED

>
```

If you see that prompt, the board is healthy and waiting for a command. Type `help` and press Enter.

```
help status led unlock
```

Notice again: `factory` is missing from the help text. Keep that in mind.

4.8 You cannot permanently brick the board

This lab is safe to experiment with. Read why.

The boot ROM cannot be erased. The RP2350 has a small read only boot program built into the chip. Even if the flash is completely erased or filled with garbage, the chip still starts that boot ROM. If you hold BOOTSEL while plugging in the USB cable, the chip enters a USB flashing mode and you can write the flash again. This recovery path always exists.

The brick routine only touches RAM. The loader's shutdown path writes a marker into a RAM word and then spins in a tight loop. It does not erase flash and it does not touch the one time programmable memory (OTP). Power cycle the board and it is back.

The probe can always halt a wedged core. Even if the firmware faults and hangs, SWD can halt the CPU and let you reflash.

The only true brick risk on this chip is modifying OTP or enabling secure boot incorrectly. This lab never does either.

So experiment freely.

Part 5: Static Recon Before Any Debugger

A good reverse engineer looks at the file before running anything. This costs nothing and often answers half the questions.

5.1 The four regions, seen in the raw bytes

```
xxd -g 4 -l 16 WEEK12/0x002a_pale-horse/build/pale-horse.bin
```

Real output:

```
00000000: 00200820 89000010 2de9f05f 4ff0ff32  . . ....-..._O..2
```

The first word is `0x20082000` and the second is `0x10000089`. That is the loader's vector table. The next word, `0x5ff0e92d`, is the first instruction: it pushes many registers onto the stack.

Now look at the main program boundary at `0x1000` in the file (which is address `0x10001000` in memory):

```
xxd -g 4 -s 0x1000 -l 16 WEEK12/0x002a_pale-horse/build/pale-horse.bin
```

Real output:

```
00001000: 00200820 09100010 00f06af9 00f086f9  . . ......j.....
```

Again you see a vector table: stack top `0x20082000` and start `0x10001009`. Then the app's first instructions.

Now the scrambled ghost at file offset `0x3e000`:

```
xxd -g 4 -s 0x3e000 -l 16 WEEK12/0x002a_pale-horse/build/pale-horse.bin
```

Real output:

```
0003e000: 2ded2e71 6dbef1db d834faf6 390a1196  -..qm....4..9...
```

Those words do not look like code. That is the point. It is scrambled.

Finally the manifest at file offset `0x3f000`:

```
xxd -g 4 -s 0x3f000 -l 48 WEEK12/0x002a_pale-horse/build/pale-horse.bin
```

Real output:

```
0003f000: 03b4c751 46554e20 534f4349 45545920  ...QFUN SOCIETY
0003f010: 414d5553 454d454e 542c204c 4c433a03  AMUSEMENT, LLC:.
0003f020: b4c7513a 37463341 ffffffff ffffffff  ..Q:7F3A........
```

Read the first four bytes little endian: `03 b4 c7 51` is `0x51c7b403`. That is the stored fingerprint. The rest is the company tag string.

5.2 Where the words `factory` and the ledger live

```
strings -t x WEEK12/0x002a_pale-horse/build/pale-horse.bin | grep -iE "factory|FUN SOCIETY|PALE-
HORSE|AUTH"
```

Real output:

```
1622 FUN SOCIETY AMUSEMENT, LLC
1657 AUTH REQUIRED
1667 Fun Society Amusement, LLC
16e1 [factory] FUN SOCIETY AMUSEMENT, LLC
1707 ASSET LEDGER PALE-HORSE-7F3A
173d factory
3f003 QFUN SOCIETY AMUSEMENT, LLC:
```

The offsets on the left are **file offsets**. To convert a file offset inside the app region to a memory address, subtract `0x1000` and add `0x10001000`. Equivalently, add `0x10000000` to the file offset for the first 256 KB, because the image is loaded at `0x10000000` and the app sits at file offset `0x1000` which is memory `0x10001000`. For example, file offset `0x173d` is memory `0x1000173d`, which is the string `factory`.

Three facts are already visible from static analysis alone:

1. The string `factory` exists at `0x1000173d`.
2. The help text does not mention it.
3. There is an asset ledger line that names the operation.

That is the entire purpose of static recon: get the big picture for free.

Part 6: Dynamic Analysis with OpenOCD and GDB, Step by Step

This part is the heart of the week. You will start the debug server, attach GDB, and watch the firmware boot one instruction at a time. Every command below shows the real response from the board.

6.0 How the debugger actually works

Three facts make everything else make sense.

SWD is a separate bus. The Debug Probe is not using the serial port or USB. It speaks a two wire protocol called SWD directly to the CPU's debug logic. That logic can halt the CPU, read and write registers, and read and write memory even while normal code is not running.

Memory reads go through the bus, not through your program. When you ask GDB to read `0x10000000`, the probe performs a bus read at that address. You do not need symbols and you do not need the program to cooperate.

Flash breakpoints must be hardware breakpoints. A normal breakpoint works by replacing the instruction at the target address with a special "trap" instruction. You cannot do that in flash without erasing a whole page, and flash cannot be written while code is running from it. So on this board you use `hbreak`, which programs one of the CPU's hardware comparators. The RP2350 Cortex-M33 has 8 of them.

6.1 Start OpenOCD

Open a terminal. Start the debug server:

```
~/openocd/src/openocd -s ~/openocd/tcl \
  -f interface/cmsis-dap.cfg \
  -f target/rp2350.cfg \
  -c "adapter speed 1000"
```

Real output, trimmed:

```
Open On-Chip Debugger 0.12.0+dev-gcf9c0b41c (2025-02-16-15:07)
Info : Hardware thread awareness created
cortex_m reset_config sysresetreq
adapter speed: 1000 kHz
Info : Listening on port 6666 for tcl connections
Info : Listening on port 4444 for telnet connections
Info : Using CMSIS-DAPv2 interface with VID:PID=0x2e8a:0x000c, serial=E6647C740371992F
Info : CMSIS-DAP: SWD supported
Info : CMSIS-DAP: Atomic commands supported
Info : CMSIS-DAP: FW Version = 2.0.0
Info : CMSIS-DAP: Interface Initialised (SWD)
Info : clock speed 1000 kHz
Info : SWD DPIDR 0x4c013477
Info : [rp2350.dap.core0] Cortex-M33 r1p0 processor detected
Info : [rp2350.dap.core0] target has 8 breakpoints, 4 watchpoints
Info : [rp2350.dap.core1] Cortex-M33 r1p0 processor detected
Info : [rp2350.dap.core1] target has 8 breakpoints, 4 watchpoints
Info : starting gdb server for rp2350.dap.core0 on 3333
Info : Listening on port 3333 for gdb connections
```

Read these lines carefully.

- `serial=E6647C740371992F` is the probe. Yours differs.
- `Cortex-M33 r1p0 processor detected` means the target answered.
- `target has 8 breakpoints, 4 watchpoints` is why we can only set a few hardware breakpoints at once.
- `Listening on port 3333` is where GDB will connect.

Leave this terminal running. Do not close it.

6.2 Start GDB and connect

Open a second terminal. Start GDB with no file. We are investigating a raw binary, so we start with no symbols on purpose:

```
arm-none-eabi-gdb
```

At the `(gdb)` prompt, connect:

```
(gdb) target remote :3333
```

Real output:

```
warning: No executable has been specified and target does not support
determining executable automatically.  Try using the "file" command.
0x10001234 in ?? ()
```

What this means:

- `0x10001234 in ?? ()` means GDB stopped the CPU at address `0x10001234` and has no name for it. The `??` is correct. We have no symbols.
- The address `0x10001234` is inside the UART input routine. The board was sitting there waiting for a serial character. That is exactly where an idle console waits.

`target remote` is the command for a probe server. Some tutorials use `target extended-remote`. Both work here. We use `target remote` for a plain halt and inspect session.

6.3 Reset and halt

```
(gdb) monitor reset halt
```

`monitor` sends a command to OpenOCD instead of to GDB. `reset halt` resets the chip and stops it immediately.

Real output:

```
[rp2350.dap.core1] VECTRESET is not supported on this Cortex-M core, using SYSRESETREQ instead.
[rp2350.dap.core1] Set 'cortex_m reset_config sysresetreq'.
[rp2350.dap.core0] halted due to debug-request, current mode: Thread
xPSR: 0xf9000000 pc: 0x00000088 msp: 0xf0000000
```

This is very informative. OpenOCD tells you the CPU halted with `pc: 0x00000088`. That is in the boot ROM. The boot ROM is about to read the vector table and jump into your firmware.

The warning about `VECTRESET` is normal on this chip. It is not an error.

6.4 Read the vector tables

Memory reads always work. Ask for the first two words of the loader and of the app:

```
(gdb) x/2xw 0x10000000
(gdb) x/2xw 0x10001000
```

Real output:

```
0x10000000:     0x20082000      0x10000089
0x10001000:     0x20082000      0x10001009
```

Interpretation:

- The loader's stack top is `0x20082000` and its start is `0x10000089`.

- The app's stack top is the same and its start is `0x10001009`.
- Both start addresses are odd, so both are Thumb code. Clear the low bit to get the real address: `0x10000088` and `0x10001008`.

`x` means examine. `2xw` means two values, in hexadecimal, each a word (4 bytes).

6.5 Stop at the loader entry

Set a hardware breakpoint at the loader's real first instruction:

```
(gdb) hbreak *0x10000088
(gdb) continue
```

Real output:

```
Hardware assisted breakpoint 1 at 0x10000088
Thread 1 "rp2350.dap.core0" hit Breakpoint 1, 0x10000088 in ?? ()
```

You are now stopped at the first instruction of the loader that you found by reading the vector table. You did this with no symbols.

The `*` before the address is required. Without it, GDB may treat the value as a function name.

6.6 Disassemble the loader

```
(gdb) x/16i $pc
```

`16i` means sixteen instructions starting at `$pc`.

Real output:

```
=> 0x10000088:  bl      0x10000130
   0x1000008c:  bl      0x10000150
   0x10000090:  ldr     r0, [pc, #116]  @ (0x10000108)
   0x10000092:  ldr     r1, [r0, #0]
   0x10000094:  ldr     r2, [pc, #116]  @ (0x1000010c)
   0x10000096:  cmp     r1, r2
   0x10000098:  beq.n   0x100000f4
   0x1000009a:  ldr     r1, [pc, #116]  @ (0x10000110)
   0x1000009c:  str     r1, [r0, #0]
   0x1000009e:  ldr     r4, [pc, #116]  @ (0x10000114)
   0x100000a0:  ldr     r5, [pc, #116]  @ (0x10000118)
   0x100000a2:  mov.w   r6, #0
   0x100000a6:  cmp     r4, r5
   0x100000a8:  bcs.n   0x100000b0
   0x100000aa:  str.w   r6, [r4], #4
   0x100000ae:  b.n     0x100000a6
```

Read this like a story.

- `bl 0x10000130` calls a helper, which sets up the stack.
- `bl 0x10000150` calls a helper, which enables the coprocessor.
- The next lines read a word, compare it to a constant, and branch to `0x100000f4` if equal. That is the boot reason check.
- Then it stores a cookie, sets up two pointers, and loops writing zero. That is the RAM scrub.

You have not run any of it yet. You have read it.

6.7 Step two instructions

```
(gdb) si
(gdb) si
(gdb) info registers pc
```

Real output:

```
0x10000130 in ?? ()
0x10000132 in ?? ()
pc             0x10000132          0x10000132
```

Interpretation: `si` is **step instruction**. The first `si` followed the call into the helper at `0x10000130`. The second `si` executed one instruction inside the helper, stopping at `0x10000132`. This is how you walk code with no symbols: one instruction at a time and read the registers.

6.8 Stop at the fingerprint routine

The first `bl` in the loader was to `0x10000130`. The fingerprint routine is called later, at `0x100000b8`. Its entry is `0x10000008`. Set a breakpoint there:

```
(gdb) hbreak *0x10000008
(gdb) continue
```

Real output:

```
Hardware assisted breakpoint 2 at 0x10000008
Thread 1 "rp2350.dap.core0" hit Breakpoint 2, 0x10000008 in ?? ()
```

Now read the arguments. On ARM, the first argument is in `r0` and the second is in `r1`:

```
(gdb) info registers pc r0 r1
```

Real output:

```
pc             0x10000008          0x10000008
r0             0x10001000          268439552
r1             0x4000              16384
```

Interpretation:

- `r0 = 0x10001000` is where the app starts.
- `r1 = 0x4000` is how many bytes to fingerprint (16384 bytes).
- So this routine computes a fingerprint over the entire app region.

Decimal values are shown next to the hexadecimal ones by GDB automatically.

6.9 Let the fingerprint finish and read the answer

```
(gdb) finish
```

`finish` runs until the current function returns, then stops. Real output:

```
Run till exit from #0  0x10000008 in ?? ()
0x100000bc in ?? ()
```

Now the answer is in `r0`:

```
(gdb) printf "return value r0 = 0x%08x\n", $r0
```

Real output:

```
return value r0 = 0x51c7b403
```

Compare it to the stored value in the manifest:

```
(gdb) x/1xw 0x1003f000
```

Real output:

```
0x1003f000:     0x51c7b403
```

They match. That is why the board booted. If they did not match, the loader would have jumped to its shutdown path. You just proved Lock 1 with real evidence.

6.10 Stop at the unscramble routine

```
(gdb) hbreak *0x10000054
(gdb) continue
```

Real output:

```
Hardware assisted breakpoint 3 at 0x10000054
Thread 1 "rp2350.dap.core0" hit Breakpoint 3, 0x10000054 in ?? ()
```

Read the four arguments:

```
(gdb) info registers pc r0 r1 r2 r3
```

Real output:

```
pc             0x10000054          0x10000054
r0             0x1003e000          268689408
r1             0x20040000          537133056
r2             0x1000              4096
r3             0x50414c45          1346456645
```

Interpretation:

- `r0 = 0x1003e000` is the scrambled source in flash.
- `r1 = 0x20040000` is the destination in RAM.
- `r2 = 0x1000` is 4096 bytes.
- `r3 = 0x50414c45` is the key. As ASCII little endian, the low bytes are `45 4c 41 50`, which is `E L A P`. Reversed, that spells `PALE`.

Look at the destination before the routine runs:

```
(gdb) x/4xw 0x20040000
```

Real output:

```
0x20040000:     0x00000000      0x00000000      0x00000000      0x00000000
```

The loader scrubbed this area first. That is the RAM scrub you read in 6.6.

6.11 Run the unscramble and watch RAM change

```
(gdb) finish
(gdb) x/4xw 0x20040000
```

Real output:

```
Run till exit from #0  0x10000054 in ?? ()
0x100000d2 in ?? ()
0x20040000:     0x0fb1a568      0x6a81481f      0xea814a1f      0x29000102
```

The zeros are gone. Real bytes appeared. But notice the first word is `0x0fb1a568`, not a valid instruction. Why?

Because the loader is not done. The very next code, at `0x100000d2`, is the **self patch**. It XORs the first word with `0x50414c45`. Compute it yourself:

```
0x0fb1a568 XOR 0x50414c45 = 0x5ff0e92d
```

`0x5ff0e92d` is the real first instruction: push many registers. The scrambled blob intentionally stores the first word masked so that a naive dump of the blob is not valid code. The loader unmasks it after the copy.

This is why the story says the program "patches one word". You just watched it get unscrambled, and you can compute the next value by hand.

6.12 Watch the jump into RAM

Now stop at the instruction sequence that leaves flash and enters RAM.

```
(gdb) hbreak *0x10001058
(gdb) continue
```

Real output:

```
Hardware assisted breakpoint 2 at 0x10001058
Thread 1 "rp2350.dap.core0" hit Breakpoint 2, 0x10001058 in ?? ()
```

Look at what is about to run:

```
(gdb) x/3i $pc
```

Real output:

```
=> 0x10001058:  ldr     r0, [pc, #20]   @ (0x10001070)
   0x1000105a:  orr.w   r0, r0, #1
   0x1000105e:  blx     r0
```

Interpretation:

- `ldr r0,[pc,#20]` loads the literal at `0x10001070`. That literal is `0x20040000`.
- `orr.w r0,r0,#1` sets the low bit, making `0x20040001`. On Cortex-M an odd branch target means Thumb.
- `blx r0` branches to that address and links.

Step two instructions to get to the branch, then step into it:

```
(gdb) si
(gdb) si
(gdb) info registers pc r0
(gdb) si
(gdb) info registers pc
```

Real output, condensed:

```
0x1000105a in ?? ()
0x1000105e in ?? ()
pc             0x1000105e          0x1000105e
r0             0x20040001          537133057
0x20040000 in ?? ()
pc             0x20040000          0x20040000
```

`pc` is now `0x20040000`. The CPU is executing code inside RAM. That code is not in the flash image as valid instructions. You are watching the hidden program run.

This is the clearest possible proof of Lock 2. You did not read about it. You saw `pc` change to RAM.

6.13 Look inside the ghost while it runs

```
(gdb) x/24i 0x20040000
```

Real output:

```
=> 0x20040000:  stmdb   sp!, {r4, r5, r6, r7, r8, r9, r10, r11, r12, lr}
   0x20040004:  ldr     r0, [pc, #124]  @ (0x20040084)
   0x20040006:  ldr     r1, [r0, #40]   @ 0x28
   0x20040008:  ldr     r2, [pc, #124]  @ (0x20040088)
   0x2004000a:  eor.w   r1, r1, r2
   0x2004000e:  cmp     r1, #0
   0x20040010:  bne.n   0x20040016
   0x20040012:  mov.w   r1, #1
   0x20040016:  ldr     r0, [pc, #116]  @ (0x2004008c)
   0x20040018:  str     r1, [r0, #0]
   0x2004001a:  ldr     r0, [pc, #116]  @ (0x20040090)
   0x2004001c:  mov.w   r1, #0
   0x20040020:  str     r1, [r0, #0]
   0x20040022:  ldmia.w sp!, {r4, r5, r6, r7, r8, r9, r10, r11, r12, lr}
   0x20040026:  bx      lr
   0x20040028:  stmdb   sp!, {r4, r5, r6, r7, r8, r9, r10, r11, r12, lr}
   0x2004002c:  ldr     r0, [pc, #92]   @ (0x2004008c)
   0x2004002e:  ldr     r1, [r0, #0]
   0x20040030:  eor.w   r2, r1, r1, lsl #13
   0x20040034:  eor.w   r2, r2, r2, lsr #17
   0x20040038:  eor.w   r2, r2, r2, lsl #5
   0x2004003c:  str     r2, [r0, #0]
   0x2004003e:  mov     r0, r2
   0x20040040:  ldmia.w sp!, {r4, r5, r6, r7, r8, r9, r10, r11, r12, lr}
```

This is the ghost. In Part 11 we will go through it line by line. For now, notice the shape: it reads a value at `0x400b0028`, XORs it with a constant, seeds a state variable, and then a shift and XOR sequence that looks like a random number generator.

6.14 The console reader and its bounds check

Halt wherever the board is and look at the reader. At a fresh prompt, or after `monitor reset halt` and a boot, the console blocks inside `UART0_In`. Look at the reader entry:

```
(gdb) monitor halt
(gdb) x/2i 0x10001090
```

Real output:

```
   0x10001090:  stmdb   sp!, {r4, r5, r6, r7, r8, r9, r10, r11, r12, lr}
   0x10001094:  ldr     r4, [pc, #152]  @ (0x10001130)
```

Now the loop and the bounds check:

```
(gdb) x/12i 0x1000109a
```

Real output:

```
0x1000109a:  bl      0x10001230
0x1000109e:  and.w   r0, r0, #255    @ 0xff
0x100010a2:  cmp     r0, #13
0x100010a4:  beq.n   0x100010ba
0x100010a6:  cmp     r0, #10
0x100010a8:  beq.n   0x100010ba
0x100010aa:  mov.w   r6, #32
0x100010ae:  cmp     r5, r6
0x100010b0:  bhi.n   0x100010ba
0x100010b2:  strb    r0, [r4, r5]
0x100010b4:  add.w   r5, r5, #1
0x100010b8:  b.n     0x1000109a
```

Interpretation, line by line:

- `bl 0x10001230` reads one character from UART0.
- `and.w r0,r0,#255` keeps the low byte, the character.
- Compare against 13 (carriage return) and 10 (line feed). Either one ends the line.
- `mov.w r6,#32` sets the limit to 32.
- `cmp r5,r6` compares the current count to 32.
- `bhi.n 0x100010ba` branches only if the count is **higher** than 32. This is the bug. It should have stopped when the count reached 32. Because it uses "higher", a count of exactly 32 is still allowed, and one more character is stored.
- `strb r0,[r4,r5]` stores the character at buffer plus count.
- `add.w r5,r5,#1` increments the count.

There it is: an off-by-one. The instruction `bhi` is the proof. You read it from the device.

6.15 Prove the off-by-one with the real store instruction

The console cannot continue without serial input, so we drive the exact vulnerable store ourselves with GDB. This is a standard reverse engineering technique: if you cannot feed input through the normal path, set up the registers and execute the instruction under the debugger.

The buffer is at `0x20000200` and is 32 bytes. The word immediately after it is at `0x20000220`. Set up a full buffer and a non-zero neighbor, then execute the terminator store:

```
(gdb) set $r4 = 0x20000200
(gdb) set $r5 = 32
(gdb) set $r0 = 0
(gdb) set {int}0x20000220 = 0x41414141
(gdb) x/1xw 0x20000220
```

Real output:

```
0x20000220:     0x41414141
```

Now jump to the terminator store, confirm the instructions, and step them:

```
(gdb) set $pc = 0x100010ba
(gdb) x/2i $pc
```

Real output:

```
=> 0x100010ba:  mov.w   r0, #0
   0x100010be:  strb    r0, [r4, r5]
```

Execute the two instructions and re-read the neighbor:

```
(gdb) si
(gdb) si
(gdb) x/1xw 0x20000220
```

Real output:

```
0x100010be in ?? ()
0x100010c0 in ?? ()
0x20000220:     0x41414100
```

The word changed from `0x41414141` to `0x41414100`. The terminator byte `0x00` was written one byte past the 32 byte buffer. That is the overflow, proven on the real device.

6.16 Dispatch the hidden command

The dispatcher walks a table. Dump it:

```
(gdb) x/10xw 0x10001748
```

Real output:

```
0x10001748:     0x10001726      0x10001145      0x1000172b      0x10001155
0x10001758:     0x10001732      0x10001165      0x10001736      0x1000117d
0x10001768:     0x1000173d      0x1000118d
```

Each pair is a name address and a handler address. The low bit on the handler means Thumb. Decode the pairs by reading the strings:

Name address	Handler address	Name
`0x10001726`	`0x10001144`	`help`
`0x1000172b`	`0x10001154`	`status`
`0x10001732`	`0x10001164`	`led`
`0x10001736`	`0x1000117c`	`unlock`
`0x1000173d`	`0x1000118c`	`factory`

There it is: `factory` is the fifth entry in the live table. It is present, just hidden from the help text.

Now prove the dispatcher will call it. Write the string `factory` into the buffer and make the dispatcher run:

```
(gdb) set {char[8]}0x20000200 = "factory"
(gdb) x/8xb 0x20000200
```

Real output:

```
0x20000200:     0x66    0x61    0x63    0x74    0x6f    0x72    0x79    0x00
```

Those bytes are `f a c t o r y` and a terminator. Set a breakpoint on the factory handler and jump to the dispatcher:

```
(gdb) hbreak *0x1000118c
(gdb) set $pc = 0x100010e8
(gdb) continue
```

Real output:

```
Hardware assisted breakpoint 1 at 0x1000118c
Thread 1 "rp2350.dap.core0" hit Breakpoint 1, 0x1000118c in ?? ()
```

The dispatcher matched `factory` from the table and branched to its handler. That is Lock 3's hidden door, proven.

6.17 Why `break` fails and `hbreak` works

If you try a plain `break *0x10000088` instead of `hbreak`, GDB will try to insert a software breakpoint. On flash this either fails or silently does nothing useful, because writing the trap instruction requires modifying flash. Then GDB reports:

```
Note: automatically using hardware breakpoints for read-only addresses.
```

GDB is telling you it had to switch to a hardware breakpoint because the address is read only. On this target, be explicit and use `hbreak` from the start. It avoids confusion and it is the correct tool for flash.

6.18 GDB command reference

Command	What it does
`target remote :3333`	attach to the OpenOCD gdb server
`monitor reset halt`	reset the chip and stop it
`monitor halt`	stop the running chip
`x/2xw 0x10000000`	read two words in hex
`x/16i $pc`	disassemble sixteen instructions at pc
`hbreak *0xADDR`	hardware breakpoint at an address
`continue`	run until a breakpoint
`si`	step one instruction, following calls
`ni`	step one instruction, not following calls
`finish`	run until the current function returns

Command	What it does
`info registers`	show all registers
`info registers pc r0 r1`	show selected registers
`set $r5 = 32`	write a register
`set {int}0x20000220 = 0x41414141`	write four bytes to memory
`set {char[8]}0x20000200 = "factory"`	write a string to memory
`set $pc = 0x100010e8`	move the program counter
`printf "0x%08x\n", $r0`	print a register as hex
`detach`	release the target and leave it running
`quit`	leave GDB

Two more that are useful when a target misbehaves:

Command	What it does
`monitor rp2350.dap.core0 configure -rtos none`	stop OpenOCD from trying to unwind an RTOS, which fixes false step reports
`set pagination off`	stop GDB pausing at page breaks, useful in scripts

Part 7: The Loader, Function by Function

This part takes the real disassembly from the shipped image and explains every function. The addresses and instructions are verbatim from the binary. The comments are the explanation.

7.1 Init_Stack at 0x10000130

```
10000130 <Init_Stack>:
10000130:  ldr  r0, [pc, #20]     @ loads 0x20082000
10000132:  msr  PSP, r0           @ process stack pointer = 0x20082000
10000136:  ldr  r0, [pc, #20]     @ loads 0x2007a000
10000138:  msr  MSPLIM, r0        @ main stack limit = 0x2007a000
1000013c:  msr  PSPLIM, r0        @ process stack limit = 0x2007a000
10000140:  ldr  r0, [pc, #4]
10000142:  msr  MSP, r0           @ main stack pointer = 0x20082000
10000146:  bx   lr
```

What it does: it points both stack pointers at the top of RAM (`0x20082000`) and sets a lower limit (`0x2007a000`). The limit makes the CPU fault if the stack grows too far. That is a basic safety measure.

7.2 Enable_Coprocessor at 0x10000150

```
10000150 <Enable_Coprocessor>:
10000150:  ldr  r0, [pc, #20]     @ loads 0xe000ed88 (CPACR)
10000152:  ldr  r1, [r0, #0]
10000154:  orr.w r1, r1, #2
10000158:  orr.w r1, r1, #1       @ enable coprocessor access
1000015c:  str  r1, [r0, #0]
1000015e:  dsb  sy
10000162:  isb  sy
10000166:  bx   lr
```

What it does: it enables access to the floating point and coprocessor hardware by setting bits in the Coprocessor Access Control Register. On the RP2350 this is needed before some instructions that touch the fast GPIO hardware.

7.3 Stage2_RESET at 0x10000088

```
10000088 <Stage2_Reset>:
10000088:  bl   10000130 <Init_Stack>
1000008c:  bl   10000150 <Enable_Coprocessor>
10000090:  ldr  r0, [pc, #116]    @ loads 0x400d800c (WATCHDOG_SCRATCH0)
10000092:  ldr  r1, [r0, #0]      @ read the boot cookie
10000094:  ldr  r2, [pc, #116]    @ loads 0xbadf00d0 (TAMPER cookie)
10000096:  cmp  r1, r2
10000098:  beq.n 100000f4         @ if tamper, shut down
1000009a:  ldr  r1, [pc, #116]    @ loads 0x1a2b3c4d (IN PROGRESS cookie)
1000009c:  str  r1, [r0, #0]      @ mark that we are mid boot
1000009e:  ldr  r4, [pc, #116]    @ loads 0x20040000 (ghost RAM start)
100000a0:  ldr  r5, [pc, #116]    @ loads 0x20041000 (ghost RAM end)
100000a2:  mov.w r6, #0
100000a6:  cmp  r4, r5
100000a8:  bcs.n 100000b0
100000aa:  str.w r6, [r4], #4     @ scrub ghost RAM to zero
100000ae:  b.n  100000a6
100000b0:  mov.w r0, #0x10001000  @ app start
100000b4:  mov.w r1, #0x4000      @ app size
100000b8:  bl   10000008 <Stage2_CRC32>
100000bc:  ldr  r2, [pc, #92]     @ loads 0x1003f000 (manifest)
100000be:  ldr  r2, [r2, #0]      @ read stored CRC
100000c0:  cmp  r0, r2
100000c2:  bne.n 100000f4         @ mismatch, shut down
100000c4:  ldr  r0, [pc, #88]     @ loads 0x1003e000 (ghost blob)
100000c6:  ldr  r1, [pc, #76]     @ loads 0x20040000 (ghost RAM)
100000c8:  mov.w r2, #0x1000      @ 4096 bytes
100000cc:  ldr  r3, [pc, #84]     @ loads 0x50414c45 (key)
100000ce:  bl   10000054 <Stage2_Xor>
100000d2:  ldr  r0, [pc, #64]     @ loads 0x20040000
100000d4:  ldr  r1, [r0, #0]      @ read first ghost word
100000d6:  ldr  r2, [pc, #76]     @ loads 0x50414c45
100000d8:  eor.w r1, r1, r2       @ unmask first word
100000dc:  str  r1, [r0, #0]
100000de:  dsb  sy
100000e2:  isb  sy
100000e6:  ldr  r0, [pc, #32]     @ loads 0x400d800c
100000e8:  ldr  r1, [pc, #60]     @ loads 0x600dbeef (OK cookie)
100000ea:  str  r1, [r0, #0]
100000ec:  mov.w r0, #0x10001000
100000f0:  ldr  r1, [r0, #4]      @ app vector word 2 = start address
100000f2:  bx   r1
```

This is the whole loader. Read it as the nine steps from Part 1: set up, read boot reason, arm the cookie, scrub, fingerprint, unscramble, self patch, clear the cookie, jump.

The boot cookie is stored in the watchdog scratch register at `0x400d800c`. That register survives a watchdog reset, which is how the loader can tell whether the last boot ended badly.

7.4 Stage2_Brick at 0x100000fa

```
100000fa <Stage2_Brick>:
100000fa:  stmdb sp!, {r4, r5, r6, r7, r8, r9, sl, fp, ip, lr}
100000fe:  ldr  r0, [pc, #20]     @ loads 0x20040000
10000100:  ldr  r1, [pc, #40]     @ loads 0xdeadb001
10000102:  str  r1, [r0, #0]      @ write the marker to RAM
10000104:  b.n  10000104          @ spin forever
```

This is the "shutdown" path. It writes the marker `0xdeadb001` into RAM at `0x20040000` and then loops on itself forever. It does not erase flash. It does not touch OTP. Power cycle the board and it is back. This is why you cannot permanently break it.

Part 8: Lock 1, the CRC-32 Integrity Gate

8.1 The algorithm

The routine at `0x10000008` is a standard CRC-32. It does three things per byte:

1. mix the byte into a running value with XOR,
2. shift the value right eight times, and when a one falls off the end, XOR in a constant,
3. at the end, invert every bit.

The constant is `0xEDB88320`, the standard reflected CRC-32 polynomial.

8.2 The real code

```
10000008 <Stage2_CRC32>:
10000008:  stmdb sp!, {r4, r5, r6, r7, r8, r9, sl, fp, ip, lr}
1000000c:  mov.w r2, #0xffffffff  @ running value = 0xFFFFFFFF
10000010:  mov   r4, r0           @ r4 = data pointer
10000012:  add.w r5, r0, r1       @ r5 = end pointer
10000016:  cmp   r4, r5
10000018:  bcs.n 10000044         @ done
1000001a:  ldrb  r3, [r4, #0]     @ next byte
1000001c:  eor.w r2, r2, r3       @ mix
10000020:  mov.w r6, #8           @ eight bits
10000024:  tst.w r2, #1
10000028:  beq.n 10000036         @ low bit zero, just shift
1000002a:  mov.w r2, r2, lsr #1   @ shift right
1000002e:  ldr   r7, [pc, #32]    @ loads 0xEDB88320
10000030:  eor.w r2, r2, r7       @ mix the polynomial
10000034:  b.n   1000003a
10000036:  mov.w r2, r2, lsr #1   @ shift right
1000003a:  subs  r6, r6, #1
1000003c:  bne.n 10000024         @ eight times
1000003e:  add.w r4, r4, #1       @ next byte
10000042:  b.n   10000016
10000044:  mvn.w r0, r2           @ invert every bit
10000048:  ldmia.w sp!, {r4, r5, r6, r7, r8, r9, sl, fp, ip, lr}
1000004c:  bx    lr
```

8.3 Prove the value by hand

The loader computes the CRC over `0x10001000` for `0x4000` bytes. That is the entire app region. It compares the result to the word at `0x1003f000`.

You can reproduce the exact value from the file with plain Python. Write this to a file and run it:

```
import zlib

data = open("WEEK12/0x002a_pale-horse/build/pale-horse.bin", "rb").read()
app = data[0x1000:0x1000 + 0x4000]
crc = zlib.crc32(app) & 0xFFFFFFFF
stored = int.from_bytes(data[0x3f000:0x3f004], "little")
print("computed", hex(crc))
print("stored  ", hex(stored))
```

Real output:

```
computed 0x51c7b403
stored   0x51c7b403
```

This matches the value you saw live in GDB. Three independent methods, the GDB read, the decompiler, and Python, all agree. That is what proof looks like.

8.4 How to beat it

If you change the app, the CRC changes completely. Compute the new CRC and write it to `0x1003f000` in your image before flashing, or write it into flash directly. Then the gate passes.

The lesson: a checksum is only as strong as the place you store it. Here, the stored value is in the same flash that you control.

Part 9: Lock 2, the XOR Scrambler and the Self Patch

9.1 The algorithm

The routine at `0x10000054` XORs each byte with a key. After each byte it advances the key:

```
key = rotate_left(key, 5) + 0x9E
```

XOR undoes itself, so the same routine both scrambles and unscrambles.

9.2 The real code

```
10000054 <Stage2_Xor>:
10000054:  stmdb sp!, {r4, r5, r6, r7, r8, r9, sl, fp, ip, lr}
10000058:  mov   r4, r0            @ r4 = source
1000005a:  mov   r5, r1            @ r5 = destination
1000005c:  mov   r6, r2            @ r6 = length
1000005e:  mov   r7, r3            @ r7 = key
10000060:  cmp   r6, #0
10000062:  beq.n 10000082          @ done
10000064:  ldrb  r0, [r4, #0]      @ scrambled byte
10000066:  eor.w r0, r0, r7        @ xor with key
1000006a:  strb  r0, [r5, #0]      @ store real byte
1000006c:  mov.w r7, r7, ror #27   @ rotate left five bits
10000070:  mov.w r0, #0x9e
10000074:  add   r7, r0            @ add 0x9E
10000076:  add.w r4, r4, #1
1000007a:  add.w r5, r5, #1
1000007e:  subs  r6, r6, #1
10000080:  b.n   10000060
10000082:  ldmia.w sp!, {r4, r5, r6, r7, r8, r9, sl, fp, ip, lr}
10000086:  bx    lr
```

Note `ror #27`. Rotating right by 27 is the same as rotating left by 5. That is why the decompiler showed `param_4 >> 0x1b | param_4 << 5`.

9.3 The self patch

After the copy, the loader does this at `0x100000d2`:

```
100000d2:  ldr  r0, [pc, #64]     @ 0x20040000
100000d4:  ldr  r1, [r0, #0]      @ first word
100000d6:  ldr  r2, [pc, #76]     @ 0x50414c45
100000d8:  eor.w r1, r1, r2
100000dc:  str  r1, [r0, #0]      @ write it back
100000de:  dsb  sy
100000e2:  isb  sy
```

The first word of the scrambled blob is stored masked. The loader unmasks it here. You proved this in GDB: the word was `0x0fb1a568` right after the XOR, and XOR with `0x50414c45` gives `0x5ff0e92d`, a valid push instruction.

The `dsb` and `isb` are required. `dsb` waits for the memory write to complete. `isb` flushes the instruction pipeline so the CPU fetches the new bytes instead of the old ones. Without these, the CPU might execute stale code.

9.4 Prove the decryption with Python

```
data = open("WEEK12/0x002a_pale-horse/build/pale-horse.bin", "rb").read()
blob = data[0x3e000:0x3e000 + 0x1000]
key = 0x50414C45
out = bytearray()
for b in blob:
    out.append(b ^ (key & 0xFF))
    key = ((key << 5) | (key >> 27)) & 0xFFFFFFFF
    key = (key + 0x9E) & 0xFFFFFFFF
first = int.from_bytes(out[0:4], "little") ^ 0x50414C45
print("first word after copy", hex(int.from_bytes(out[0:4], "little")))
print("first word after patch", hex(first))
```

Real output:

```
first word after copy 0xfb1a568
first word after patch 0x5ff0e92d
```

That matches the RAM you saw in GDB, and the patched word is the real ghost entry instruction.

Part 10: Lock 3, the Console

10.1 Console_PrintStr at 0x10001074

```
10001074 <Console_PrintStr>:
10001074:  stmdb sp!, {r4, r5, r6, r7, r8, r9, sl, fp, ip, lr}
10001078:  mov   r4, r0            @ string pointer
1000107a:  ldrb  r0, [r4]          @ next character
1000107c:  cmp   r0, #0
1000107e:  beq.n 1000108a          @ end of string
10001080:  bl    10001212 <UART0_Out>
10001084:  add   r4, r4, #1
10001088:  b.n   1000107a
1000108a:  ldmia.w sp!, {...}
1000108e:  bx    lr
```

A simple string printer. It walks the string and sends each character to `UART0_Out`.

10.2 Console_ReadLine at 0x10001090

```
10001090 <Console_ReadLine>:
10001090:  stmdb sp!, {...}
10001094:  ldr   r4, [pc, #152]    @ r4 = cmd_buf = 0x20000200
10001096:  mov.w r5, #0            @ count = 0
1000109a:  bl    10001230 <UART0_In>
1000109e:  and.w r0, r0, #0xff
100010a2:  cmp   r0, #13
100010a4:  beq.n 100010ba          @ carriage return ends the line
100010a6:  cmp   r0, #10
100010a8:  beq.n 100010ba          @ line feed ends the line
100010aa:  mov.w r6, #32           @ limit
100010ae:  cmp   r5, r6
100010b0:  bhi.n 100010ba          @ BUG: branch only if higher than 32
100010b2:  strb  r0, [r4, r5]      @ store character
100010b4:  add.w r5, r5, #1        @ count++
100010b8:  b.n   1000109a
100010ba:  mov.w r0, #0
100010be:  strb  r0, [r4, r5]      @ write terminator, can be one past
100010c0:  ldmia.w sp!, {...}
100010c4:  bx    lr
```

The bug is `bhi`. With `cmp r5,r6` and `bhi`, the branch is taken only when `r5` is strictly greater than 32. So when `r5` equals 32, the code continues, stores a character at `buffer[32]`, and later stores the terminator at `buffer[32]`. `buffer[32]` is one byte past a 32 byte buffer. In this build the adjacent word is `cmd_handler` at `0x20000220`.

10.3 Console_Strcmp at 0x100010c6

```
100010c6 <Console_Strcmp>:
100010c6:  ldrb  r2, [r0]
100010c8:  ldrb  r3, [r1]
100010ca:  cmp   r2, r3
100010cc:  bne.n 100010e2          @ different, return 1
100010ce:  cmp   r2, #0
100010d0:  beq.n 100010dc          @ both ended, return 0
100010d2:  add.w r0, r0, #1
100010d6:  add.w r1, r1, #1
100010da:  b.n   100010c6
100010dc:  mov.w r0, #0
100010e0:  bx    lr
100010e2:  mov.w r0, #1
100010e6:  bx    lr
```

A classic byte by byte string compare. It returns 0 if equal and 1 if different.

10.4 Console_Dispatch at 0x100010e8

```
100010e8 <Console_Dispatch>:
100010e8:  stmdb sp!, {...}
100010ec:  ldr   r4, [pc, #68]     @ r4 = cmd_table = 0x10001748
100010ee:  ldr   r5, [r4, #0]      @ name pointer
100010f0:  cmp   r5, #0
100010f2:  beq.n 1000110c          @ end of table, unknown command
100010f4:  ldr   r0, [pc, #56]     @ r0 = cmd_buf
100010f6:  mov   r1, r5
100010f8:  bl    100010c6 <Console_Strcmp>
100010fc:  cmp   r0, #0
100010fe:  beq.n 10001106          @ match found
10001100:  add.w r4, r4, #8        @ next pair
10001104:  b.n   100010ee
10001106:  ldr   r5, [r4, #4]      @ handler address
10001108:  blx   r5                @ call it
1000110a:  b.n   10001112
1000110c:  ldr   r0, [pc, #40]     @ unknown message
1000110e:  bl    10001074 <Console_PrintStr>
10001112:  ldmia.w sp!, {...}
10001116:  bx    lr
```

The dispatcher walks the table in pairs of 8 bytes. The first word is the name, the second is the handler. On a match it calls the handler with `blx`.

10.5 The command table at 0x10001748

Real dump from GDB:

```
0x10001748: 0x10001726 0x10001145
0x10001750: 0x1000172b 0x10001155
0x10001758: 0x10001732 0x10001165
0x10001760: 0x10001736 0x1000117d
0x10001768: 0x1000173d 0x1000118d
0x10001770: 0x00000000 0x00002341
```

The final zero word ends the table. The five real commands are help, status, led, unlock, and factory. Factories handler is `0x1000118c` after clearing the Thumb bit.

10.6 The command handlers

Each handler prints one string. Here is a simplified view. Each line stands for a few real instructions.

```
10001144 <Cmd_Help>:     ldr r0, =msg_help;      bl Console_PrintStr; pop; bx lr
10001154 <Cmd_Status>:   ldr r0, =msg_status;    bl Console_PrintStr; pop; bx lr
10001164 <Cmd_Led>:      mov r0,#16; bl GPIO_Set; ldr r0,=msg_led; bl Console_PrintStr; pop; bx lr
1000117c <Cmd_Unlock>:   ldr r0, =msg_unlock;    bl Console_PrintStr; pop; bx lr
1000118c <Cmd_Factory>:  ldr r0, =msg_factory;   bl Console_PrintStr; pop; bx lr
```

The factory handler prints the exact string at `0x100016e1`:

```
[factory] FUN SOCIETY AMUSEMENT, LLC
ASSET LEDGER PALE-HORSE-7F3A
```

That is the payload. It was in the binary the whole time.

10.7 The console loop at 0x10001118

```
10001118 <Console_Loop>:
10001118:  ldr r0, =msg_banner;  bl Console_PrintStr
1000111e:  ldr r0, =msg_prompt;  bl Console_PrintStr
10001124:  bl 10001090 <Console_ReadLine>
10001128:  bl 100010e8 <Console_Dispatch>
1000112c:  b.n 1000111e
```

Banner once, then forever: prompt, read a line, dispatch it, repeat.

Part 11: The Ghost

The ghost is the module that is unscrambled into RAM at `0x20040000`. It implements a small pseudo random generator and a challenge and response check.

11.1 Ghost_Entry at 0x20040000

```
20040000 <Ghost_Entry>:
20040000:  stmdb sp!, {r4, r5, r6, r7, r8, r9, r10, r11, r12, lr}
20040004:  ldr   r0, [pc, #124]    @ r0 = 0x400b0028
20040006:  ldr   r1, [r0, #40]     @ read TIMERAWL, low 32 bits of the free running timer
20040008:  ldr   r2, [pc, #124]    @ r2 = 0x50414c45
2004000a:  eor.w r1, r1, r2        @ seed = timer ^ key
2004000e:  cmp   r1, #0
20040010:  bne.n 20040016
20040012:  mov.w r1, #1            @ never allow a zero state
20040016:  ldr   r0, [pc, #116]    @ r0 = 0x200400bc (g_lfsr)
20040018:  str   r1, [r0, #0]      @ store seed
2004001a:  ldr   r0, [pc, #116]    @ r0 = 0x200400c0 (g_challenge)
2004001c:  mov.w r1, #0
20040020:  str   r1, [r0, #0]      @ challenge = 0
20040022:  ldmia.w sp!, {...}
20040026:  bx    lr
```

`0x400b0000` is the TIMER0 peripheral. Offset `0x28` is `TIMERAWL`, the low 32 bits of the free running timer. So the ghost seeds its generator from the time since power on, mixed with the constant `0x50414c45`. That is why the sequence differs from boot to boot.

11.2 Ghost_Step at 0x20040028

```
20040028 <Ghost_Step>:
20040028:  stmdb sp!, {...}
2004002c:  ldr   r0, [pc, #92]     @ 0x200400bc (g_lfsr)
2004002e:  ldr   r1, [r0, #0]
20040030:  eor.w r2, r1, r1, lsl #13
20040034:  eor.w r2, r2, r2, lsr #17
20040038:  eor.w r2, r2, r2, lsl #5
2004003c:  str   r2, [r0, #0]
2004003e:  mov   r0, r2
20040040:  ldmia.w sp!, {...}
20040044:  bx    lr
```

This is a xorshift32 generator. The shifts are 13, 17, and 5. It updates the state and returns the new value. Every call produces the next number in the sequence.

11.3 Ghost_Challenge at 0x20040046

```
20040046 <Ghost_Challenge>:
20040046:  stmdb sp!, {...}
2004004a:  bl    20040028 <Ghost_Step>   @ next random value
2004004e:  ldr   r1, [pc, #64]           @ 0x200400c0 (g_challenge)
20040050:  str   r0, [r1, #0]            @ store as the challenge
20040052:  ldmia.w sp!, {...}
20040056:  bx    lr
```

It generates the next random value and publishes it as the challenge.

11.4 Ghost_Check at 0x20040058

```
20040058 <Ghost_Check>:
20040058:  stmdb sp!, {...}
2004005c:  ldr   r1, [pc, #48]     @ 0x200400c0 (g_challenge)
2004005e:  ldr   r1, [r1, #0]      @ read challenge
20040060:  ldr   r2, [pc, #36]     @ 0x50414c45
20040062:  eor.w r1, r1, r2
20040066:  mov.w r1, r1, ror #25   @ rotate right 25
2004006a:  ldr   r2, [pc, #40]     @ 0x01000193
2004006c:  mul.w r1, r2, r1        @ multiply
20040070:  cmp   r0, r1             @ compare against the answer passed in r0
20040072:  bne.n 2004007a
20040074:  mov.w r0, #0             @ success
20040078:  b.n   2004007e
2004007a:  mov.w r0, #1             @ failure
2004007e:  ldmia.w sp!, {...}
20040082:  bx    lr
```

So the correct answer for a given challenge is:

```
answer = 0x01000193 * rotate_right_32(challenge XOR 0x50414c45, 25)
```

`0x01000193` is the FNV-1a 32 bit prime. `ror #25` is a bit rotation.

11.5 Compute an answer by hand

```
def ror(x, n):
    return ((x >> n) | (x << (32-n))) & 0xFFFFFFFF

def step(state):
    state = (state ^ (state << 13)) & 0xFFFFFFFF
    state = (state ^ (state >> 17)) & 0xFFFFFFFF
    state = (state ^ (state << 5)) & 0xFFFFFFFF
    return state

key = 0x50414C45
mult = 0x01000193
seed = 0x12345678
challenge = step(seed)
answer = (ror(challenge ^ key, 25) * mult) & 0xFFFFFFFF
print("challenge", hex(challenge))
print("answer   ", hex(answer))
```

Real output:

```
challenge 0x87985aa5
answer    0xca81f871
```

If the board issues challenge `0x87985aa5`, the answer `0xca81f871` passes the check. Because the seed comes from the free running timer, the challenge changes on every boot, so you must compute it fresh each time. That is exactly what the story means by a password that changes every time. The answer is not guessable, but it is computable once you reverse the rule.

Part 12: Static Analysis with Ghidra, Step by Step

GDB shows you what happens. Ghidra shows you what the code says. This part does the same investigation on the raw `.bin` and shows the real decompiler output.

Two rules before you start.

1. **Load the `.bin`, not the `.elf`.** The `.elf` already has names. The point is to make Ghidra name things for you.
2. **Set the base address to `0x10000000`.** The raw file has no address information, so you must tell Ghidra where it lives. On the RP2350, flash code lives at `0x10000000`.

12.1 Create the project

1. Launch Ghidra.
2. Choose `File -> New Project`.
3. Choose `Non-Shared Project` and click `Next`.
4. Name it `PALE-HORSE` and click `Finish`.

12.2 Import the raw binary

1. Choose `File -> Import File`.
2. Select `WEEK12/0x002a_pale-horse/build/pale-horse.bin`.
3. In the import dialog set **Format** to `Raw Binary`.
4. Set **Language** to `ARM:LE:32:Cortex` (little endian Cortex-M).
5. Click the options button and set **Base Address** to `0x10000000`.
6. Click `OK`, then in the summary dialog click `OK` again.
7. Double-click the imported file to open the CodeBrowser.

If you forget the base address, every address in Ghidra will be `0x0` and every reference will be wrong. This is the most common mistake.

12.3 Analyze

When Ghidra asks to analyze, click `Yes`. Accept the defaults and click `Analyze`. Analysis finds strings, references, and tries to identify functions. It may take a minute on a 256 KB image.

12.4 Tell Ghidra where the code starts

Ghidra cannot always tell that a block of bytes is Thumb code. Our code is Thumb. To force it:

1. Press `G` (Go To).
2. Type `0x10000089` and press Enter. Use the odd address from the vector table. The odd low bit tells an ARM tool that this is Thumb.
3. Press `F` (Create Function).
4. If Ghidra created the function but the instructions look wrong (four byte instructions that make no sense), right-click the function, choose `Set Thumb Mode`, or use the disassembly context menu to set the TMode context register to 1.

Once the listing shows `push` and `bl` instead of nonsense, you are in Thumb mode and the rest of the image will follow.

12.5 Name the functions

The addresses you will use are the same addresses you found with `nm` on the `.elf` and with GDB. You can confirm them with:

```
arm-none-eabi-nm -n WEEK12/0x002a_pale-horse/build/stage2.elf | grep -Ei " [tT] "
```

Real output, partial:

```
10000008 T Stage2_CRC32
10000054 T Stage2_Xor
10000088 T Stage2_Reset
100000fa T Stage2_Brick
10000130 T Init_Stack
10000150 T Enable_Coprocessor
```

In Ghidra, navigate to each address with `G` and rename it with `L`. Give the functions the names above. Renaming is not required to solve the lab, but it makes the decompiler output readable.

12.6 Decompile the CRC routine at 0x10000008

Navigate with `G` to `0x10000008`. The listing panel shows the disassembly. The right panel shows the **Decompiler**.

Real Ghidra decompiler output:

```
uint FUN_10000008(byte *param_1,int param_2)

{
  uint uVar1;
  byte *pbVar2;
  int iVar3;

  uVar1 = 0xffffffff;
  pbVar2 = param_1 + param_2;
  for (; param_1 < pbVar2; param_1 = param_1 + 1) {
    uVar1 = uVar1 ^ *param_1;
    iVar3 = 8;
    do {
      if ((uVar1 & 1) == 0) {
        uVar1 = uVar1 >> 1;
      }
      else {
        uVar1 = uVar1 >> 1 ^ _DAT_10000050;
      }
      iVar3 = iVar3 + -1;
    } while (iVar3 != 0);
  }
  return ~uVar1;
}
```

Read it in plain English.

- Start the running value at `0xFFFFFFFF`.
- For every byte, XOR it into the value.
- Do eight times: if the lowest bit is zero, shift right. Otherwise shift right and XOR the constant at `0x10000050`.
- At the end, invert every bit with `~`.

The constant `_DAT_10000050` is the value stored at `0x10000050`, which is `0xEDB88320`. You can confirm by pressing `G`, typing `0x10000050`, and reading the data.

This is a textbook CRC-32, and the decompiler recovered it correctly.

12.7 Decompile the XOR routine at 0x10000054

Real Ghidra decompiler output:

```
void FUN_10000054(byte *param_1,byte *param_2,int param_3,uint param_4)

{
  for (; param_3 != 0; param_3 = param_3 + -1) {
    *param_2 = *param_1 ^ (byte)param_4;
    param_4 = (param_4 >> 0x1b | param_4 << 5) + 0x9e;
    param_1 = param_1 + 1;
    param_2 = param_2 + 1;
  }
  return;
}
```

In plain English: destination byte equals source byte XOR the low byte of the key. Then the key is rotated and incremented. `param_4 >> 0x1b | param_4 << 5` is a rotate left by 5, written as two shifts and an OR.

12.8 Decompile the loader at 0x10000088

Real Ghidra decompiler output:

```
void FUN_10000088(void)

{
  uint *puVar1;
  int iVar2;
  uint *puVar3;

  func_0x10000130();
  func_0x10000150();
  if (*_DAT_10000108 != _DAT_1000010c) {
    *_DAT_10000108 = _DAT_10000110;
    puVar1 = _DAT_10000118;
    for (puVar3 = _DAT_10000114; puVar3 < puVar1; puVar3 = puVar3 + 1) {
      *puVar3 = 0;
    }
    iVar2 = FUN_10000008(0x10001000,0x4000);
    if (iVar2 == *_DAT_1000011c) {
      FUN_10000054(_DAT_10000120,_DAT_10000114,0x1000,_DAT_10000124);
      *_DAT_10000114 = *_DAT_10000114 ^ _DAT_10000124;
      DataSynchronizationBarrier(0xf);
      InstructionSynchronizationBarrier(0xf);
      *_DAT_10000108 = _DAT_10000128;
      (*pcRam10001004)();
      return;
    }
  }
  do {
    func_0x100000fa();
  } while( true );
}
```

This is the same loader you read in Part 7, now in C. The `if` on `_DAT_10000108` is the boot reason check. The loop that writes zero is the scrub. The call to `FUN_10000008(0x10001000,0x4000)` is the CRC. The call to `FUN_10000054(...)` is the unscramble. The `^` on the ghost first word is the self patch. The `DataSynchronizationBarrier` and `InstructionSynchronizationBarrier` are the `dsb` and `isb`. The indirect call at the end is the jump into the app.

12.9 Decompile the app entry at 0x10001008

Real Ghidra decompiler output:

```
void FUN_10001008(void)

{
  undefined4 uVar1;
  int iVar2;

  func_0x100012e0();
  func_0x1000131c();
  func_0x10001336();
  func_0x100012c0();
  func_0x10001300();
  func_0x100011b0();
  func_0x100011c6();
  uVar1 = _DAT_10001068;
  uRam20000000 = uRam10001000;
  uRam20000004 = uRam10001004;
  for (iVar2 = 2; iVar2 != 0x10; iVar2 = iVar2 + 1) {
    *(undefined4 *)(iVar2 * 4 + 0x20000000) = uVar1;
  }
  *_DAT_1000106c = 0x20000000;
  DataSynchronizationBarrier(0xf);
  InstructionSynchronizationBarrier(0xf);
  (*(code *)(_DAT_10001070 | 1))();
  FUN_10001074(PTR_s_FUN_SOCIETY_AMUSEMENT__LLC_DEAD_D_1000113c);
  do {
    FUN_10001074(PTR_DAT_10001140);
    func_0x10001090();
    func_0x100010e8();
  } while( true );
}
```

In plain English: initialize the clock and UART, copy the vector table to RAM at `0x20000000`, point the vector table register at it, call the ghost with the low bit set, then print the banner and loop forever reading and dispatching commands.

12.10 Decompile the line reader at 0x10001090 and find the bug

Real Ghidra decompiler output:

```
void FUN_10001090(void)

{
  int iVar1;
  char cVar2;
  uint uVar3;

  iVar1 = _DAT_10001130;
  uVar3 = 0;
  while( true ) {
    cVar2 = func_0x10001230();
    if (((cVar2 == '\r') || (cVar2 == '\n')) || (0x20 < uVar3)) break;
    *(char *)(iVar1 + uVar3) = cVar2;
    uVar3 = uVar3 + 1;
  }
  *(undefined1 *)(iVar1 + uVar3) = 0;
  return;
}
```

The bug is right here: `0x20 < uVar3`, that is 32 is less than the count, so the loop continues until the count is greater than 32. When the count is exactly 32 it still writes one more character. Then the terminator write at the end writes at `buffer[count]`, which can be index 32, one past a 32 byte buffer.

Ghidra recovered the exact wrong comparison in C. This is why static analysis matters: you can see the bug without typing a single character into the console.

12.11 Decompile the dispatcher at 0x100010e8

Real Ghidra decompiler output:

```
void FUN_100010e8(void)

{
  int iVar1;
  int *piVar2;

  piVar2 = (int *)PTR_PTR_10001134;
  while( true ) {
    if (*piVar2 == 0) {
      FUN_10001074(PTR_DAT_10001138);
      return;
    }
    iVar1 = func_0x100010c6(_DAT_10001130,*piVar2);
    if (iVar1 == 0) break;
    piVar2 = piVar2 + 2;
  }
  (*(code *)piVar2[1])();
  return;
}
```

In plain English: walk the table two words at a time, compare the buffer to each name, and when one matches, call the second word of that pair. The table pointer is `PTR_PTR_10001134` which points at `0x10001748`.

12.12 Decompile the hidden handler at 0x1000118c

Real Ghidra decompiler output:

```
void FUN_1000118c(void)

{
  FUN_10001074(PTR_s__factory__FUN_SOCIETY_AMUSEMENT__100011ac);
  return;
}
```

Ghidra even shows the string pointer name: `PTR_s__factory__FUN_SOCIETY_AMUSEMENT__100011ac`. The handler prints the factory banner. The string is stored at `0x100016e1`.

12.13 Decompile the ghost

The ghost is not valid code in flash, so it will not analyze there. Use the ghost ELF, which is the same code unencrypted:

1. Choose `File -> Import File`.
2. Select `WEEK12/0x002a_pale-horse/build/ghost.elf`.
3. Ghidra detects ELF and loads it at its real address, `0x20040000`.

Real Ghidra decompiler output for the entry:

```
void Ghost_Entry(void)

{
  uint uVar1;

  uVar1 = *(uint *)(DAT_20040084 + 0x28) ^ DAT_20040088;
  if (uVar1 == 0) {
    uVar1 = 1;
  }
  *DAT_2004008c = uVar1;
  *DAT_20040090 = 0;
  return;
}
```

Real output for the step:

```
uint Ghost_Step(void)

{
  uint uVar1;

  uVar1 = *DAT_2004008c ^ *DAT_2004008c << 0xd;
  uVar1 = uVar1 ^ uVar1 >> 0x11;
  uVar1 = uVar1 ^ uVar1 << 5;
  *DAT_2004008c = uVar1;
  return uVar1;
}
```

Real output for the challenge:

```
void Ghost_Challenge(void)

{
  undefined4 uVar1;

  uVar1 = Ghost_Step();
  *DAT_20040090 = uVar1;
  return;
}
```

Real output for the check:

```
bool Ghost_Check(int param_1)

{
  return param_1 !=
         DAT_20040094 *
         ((*DAT_20040090 ^ DAT_20040088) >> 0x19 | (*DAT_20040090 ^ DAT_20040088) << 7);
}
```

Ghidra recovered the entire challenge and response rule, including the multiplier and the rotation. You can read the answer formula directly: multiply the rotated XOR by the constant and compare to the value passed in.

12.14 Ghidra reference

Action	What to do
Go to an address	press `G`, type the address, press Enter

Action	What to do
Create a function	press `F` at the address
Rename a function	press `L` or right-click and choose `Rename`
Retype a variable	press `Y` (Change Type)
Show the decompiler	window menu, `Decompiler`, or `Ctrl+E`
Import a file	`File -> Import File`
Set base address	import options, `Base Address`
Search for a string	`Search -> For Strings`
List functions	`Window -> Functions`, or the Symbol Tree

Part 13: The Answers, Collected

Everything below was proven above with real output. This is the answer key.

Question	Answer	Where proven
What is the stack top?	`0x20082000`	6.4
What is the loader entry?	`0x10000088`	6.4 and 6.5
What is the app entry?	`0x10001008`	6.4 and 12.9
What is the app size for the CRC?	`0x4000` bytes from `0x10001000`	6.8
What is the stored CRC?	`0x51c7b403`	6.9 and 8.3
What is the scramble key?	`0x50414c45`	6.10
What is the ghost RAM address?	`0x20040000`	6.10
What is the ghost first word after the patch?	`0x5ff0e92d`	9.3
Where is the manifest?	`0x1003f000`	5.2 and 6.9
What is the hidden command?	`factory`	6.16 and 10.5
What does factory print?	`ASSET LEDGER PALE-HORSE-7F3A`	10.6
What is the ghost answer formula?	`0x01000193 * ror(challenge XOR 0x50414c45, 25)`	11.5 and 12.13
What is the reader bug?	`bhi` allows a count of 32, so one byte overflows	6.15 and 12.10

Part 14: The Hacks

Each hack lists the idea, the exact steps, and the real command to prove it.

14.1 Read the manifest with the debugger

You do not need any lock to read the secret list. It is in flash and the debugger reads memory directly.

```
(gdb) x/12xw 0x1003f000
```

Real output:

```
0x1003f000:     0x51c7b403      0x204e5546      0x49434f53      0x20595445
0x1003f010:     0x53554d41      0x4e454d45      0x4c202c54      0x033a434c
0x1003f020:     0x3a51c7b4      0x41334637      0xffffffff      0xffffffff
```

Read the first word as the CRC, `0x51c7b403`. The following words are the bytes of the tag. Dump the same region as bytes to read the tag directly:

```
(gdb) x/48xb 0x1003f000
```

Real output:

```
0x1003f000:     0x03    0xb4    0xc7    0x51    0x46    0x55    0x4e    0x20
0x1003f008:     0x53    0x4f    0x43    0x49    0x45    0x54    0x59    0x20
0x1003f010:     0x41    0x4d    0x55    0x53    0x45    0x4d    0x45    0x4e
0x1003f018:     0x54    0x2c    0x20    0x4c    0x4c    0x43    0x3a    0x03
0x1003f020:     0xb4    0xc7    0x51    0x3a    0x37    0x46    0x33    0x41
0x1003f028:     0xff    0xff    0xff    0xff    0xff    0xff    0xff    0xff
```

Decode the bytes after the CRC: `46 55 4e 20` is `F U N `, and the run continues to spell `FUN SOCIETY AMUSEMENT, LLC:`. The secret was never unreachable. The locks protect running the program, not reading flash.

14.2 Call the hidden command

The `factory` command is in the table. You can reach it with the serial console if you can type a 32 character line plus the overflow, or you can prove it with GDB as in 6.16:

```
(gdb) set {char[8]}0x20000200 = "factory"
(gdb) hbreak *0x1000118c
(gdb) set $pc = 0x100010e8
(gdb) continue
```

The breakpoint on the handler fires. That is the hidden door.

14.3 Use the off-by-one

Send a command line of exactly 32 characters. The reader's `bhi` lets it through. Then the terminator store writes one byte at index 32, into `cmd_handler`. You proved the exact store in 6.15. To watch it on the real console, set a breakpoint at `0x100010be` and send the long line; the write happens on the real device.

The lesson is bigger than this one byte. The bug is a missing lower-than check, and the exact instruction `bhi` is the proof. Finding the class of bug matters more than the one byte.

14.4 Recompute the CRC after a patch

If you change the app, the gate fails. Recompute and rewrite:

```
import zlib

path = "WEEK12/0x002a_pale-horse/build/pale-horse.bin"
data = bytearray(open(path, "rb").read())
app = data[0x1000:0x1000 + 0x4000]
crc = zlib.crc32(app) & 0xFFFFFFFF
data[0x3f000:0x3f004] = crc.to_bytes(4, "little")
open("WEEK12/0x002a_pale-horse/build/pale-horse-patched.bin", "wb").write(data)
print("new crc", hex(crc))
```

Real output for the unmodified app:

```
new crc 0x51c7b403
```

Change a byte in the app region and run it again, and the printed value changes. Flash the patched file and the gate accepts it. This is the integrity gate defeated correctly, not bypassed.

14.5 Compute the ghost answer

Use the formula from Part 11 with the live challenge. Read the challenge from the board or from the routine, compute the answer, and send it. Because the seed is the timer, the answer changes every boot, so always read the current challenge first.

14.6 Exercise the ghost check directly

The app calls only `Ghost_Entry` at boot, so to see the check decide pass or fail you drive it under GDB. Boot the board, stop after the ghost has been loaded, set a known challenge, and call the check with the correct answer.

```
(gdb) monitor reset halt
(gdb) hbreak *0x10001060
(gdb) continue
(gdb) set {int}0x200400c0 = 0x87985aa5
(gdb) set $r0 = 0xca81f871
(gdb) set $lr = 0x10001060
(gdb) hbreak *0x2004007e
(gdb) set $pc = 0x20040058
(gdb) continue
(gdb) info registers r0
```

Real output, condensed:

```
Hardware assisted breakpoint 2 at 0x2004007e
Thread 1 "rp2350.dap.core0" hit Breakpoint 2, 0x2004007e in ?? ()
r0             0x0                 0
```

`r0 = 0` means the check passed. Now repeat with a wrong answer `0x12345678`:

```
r0             0x1                 1
```

`r0 = 1` means the check failed. The constant `0xca81f871` was computed from the formula in Part 11 and the challenge `0x87985aa5`, and the device accepted it. That is the challenge and response defeated by understanding it, not by patching it.

14.7 The honest summary of the three locks

- Lock 1 checks a fingerprint whose value you can recompute and rewrite.
- Lock 2 hides code in RAM, but the key is in the loader and the RAM is visible to the debugger.
- Lock 3 hides a command in plain sight and has a one byte overflow.

None of them is broken cryptography. They are defenses that assume you cannot read the device. The debugger removes that assumption. That is the real lesson of the week.

Part 15: Checkpoints

Use these to confirm you are on track. Each one is a real value from the shipped image.

Check	Command	Expected
Probe found	start OpenOCD	`serial=` appears and `Cortex-M33 r1p0 processor detected`
Vector table	`x/2xw 0x10000000`	`0x20082000 0x10000089`
App vectors	`x/2xw 0x10001000`	`0x20082000 0x10001009`
CRC arguments	`info registers r0 r1` at `0x10000008`	`r0 0x10001000`, `r1 0x4000`
CRC result	`finish` then `p/x $r0`	`0x51c7b403`
Stored CRC	`x/1xw 0x1003f000`	`0x51c7b403`
XOR arguments	`info registers r0 r1 r2 r3` at `0x10000054`	`0x1003e000`, `0x20040000`, `0x1000`, `0x50414c45`
Ghost RAM after copy	`x/4xw 0x20040000` after `finish`	first word `0x0fb1a568`
Ghost RAM after patch	`x/4xw 0x20040000` after `dsb`	first word `0x5ff0e92d`
Jump into RAM	`info registers pc` after `si` at `0x1000105e`	`0x20040000`
Buffer	`x/8xb 0x20000200` after typing `factory`	`66 61 63 74 6f 72 79 00`
Command table	`x/10xw 0x10001748`	five name and handler pairs
Off-by-one	drive `0x100010be` with `r5 = 32`	neighbor word low byte becomes `0x00`
Hidden handler	`hbreak *0x1000118c` then dispatch `factory`	breakpoint fires

Part 16: Troubleshooting

Each entry below is a real problem and the real message you will see.

16.1 GDB cannot connect

If `target remote :3333` hangs or errors, the usual causes are:

- OpenOCD is not running. Start it first and wait for `Listening on port 3333`.
- Another program already owns the probe. Close any other GDB, IDE debug session, or second OpenOCD.
- The probe is not connected. Recheck SWDIO, SWCLK, and GND.

Our working toolchain was GDB 14.2. If you are on GDB 13.3 and the connection fails, move to 14.2 or later.

16.2 The board seems to be at the wrong address

After `monitor reset halt`, OpenOCD reports:

```
pc: 0x00000088
```

That is the boot ROM. GDB may print a stale value from before the reset until you resume or the target refreshes. Trust memory reads (`x`) and trust the OpenOCD message. Set a breakpoint and `continue` to reach the code you want.

16.3 Software breakpoints do nothing

If a plain `break` does not stop, or GDB says:

```
Note: automatically using hardware breakpoints for read-only addresses.
```

you are trying to place a software breakpoint in flash. Use `hbreak` instead. The RP2350 has 8 hardware breakpoints, so do not set more than a few at once.

16.4 False single steps

If `si` jumps around unpredictably, OpenOCD may be trying to unwind an RTOS context. Disable it:

```
(gdb) monitor rp2350.dap.core0 configure -rtos none
```

This was the fix for false step reports during bring up.

16.5 No serial output

The firmware prints on UART0, on GP0, at 115200. It does not print on USB.

- If you opened the Pico 2 USB serial port, you will see nothing. Connect a USB to UART adapter to GP0 and GP1.
- Check the adapter orientation: board GP0 goes to adapter RX, board GP1 goes to adapter TX, and grounds must be joined.
- Confirm 115200, 8 data bits, no parity, 1 stop bit.

The firmware side was verified: reading the UART registers showed the transmitter enabled and the transmit path draining, so the bytes are leaving the chip.

16.6 The board will not start after a change

That is the integrity gate doing its job. Recompute the CRC over `0x10001000` for `0x4000` bytes and write it to `0x1003f000`, then reflash. See 14.4.

16.7 The ghost looks like noise in Ghidra

The ghost is stored scrambled in flash. Ghidra cannot analyze it there. Import `ghost.elf`, or decode the blob with the Python in Part 9 first. The runtime RAM copy is at `0x20040000` only after the loader has run.

16.8 Ghidra shows four byte nonsense

You are in ARM mode, not Thumb mode. Navigate to the odd entry address `0x10000089`, create the function there, and set the TMode context register to 1. The listing should then show `push` and `bl`.

Part 17: Summary and Review

17.1 What the board does, in one paragraph

The boot ROM reads the vector table at `0x10000000` and jumps to the loader at `0x10000088`. The loader sets up the stack, checks a boot cookie in the watchdog scratch register, scrubs the ghost RAM area, computes a CRC-32 over the app, compares it to the value in the manifest at `0x1003f000`, unscrambles the ghost from `0x1003e000` into RAM at `0x20040000`, unmasks the first ghost word, clears the cookie, and jumps to the app at `0x10001008`. The app initializes the clock and UART, relocates its vector table into RAM, calls the ghost entry, prints a banner, and loops forever reading a command line and dispatching it through a table of five handlers, one of which is hidden.

17.2 What you did

1. Identified the four regions of the image with `xxd` and `strings`.
2. Read the vector tables with GDB and found both entry points.
3. Watched the loader run and read its arguments.
4. Proved the CRC by three independent methods: live, decompiled, and Python.
5. Watched the ghost decrypt into RAM and patched its first word.
6. Stepped across the branch from flash into RAM and saw `pc` become `0x20040000`.
7. Found the off-by-one in the reader and drove the exact store instruction to prove it.
8. Dumped the live command table and dispatched the hidden `factory` command.
9. Recovered the full challenge and response rule with Ghidra and computed a valid answer.
10. Explained why none of this can permanently brick the board.

17.3 Why each lock failed

Lock	The assumption it makes	Why the assumption is false
CRC gate	you cannot change the program	you can change it and recompute the CRC
hidden room	you only look at flash	the debugger reads RAM while it runs
console	you cannot see the table or the bug	the table and the bug are in the binary

The common thread: each defense assumes the attacker cannot read the device. A debugger removes that assumption.

Key Takeaways

1. A `.bin` is the program itself. A `.elf` is the same program plus names and a map. Investigating the `.bin` is the realistic exercise.
2. The first two words are the vector table, and an odd entry address means Thumb code. Clear the low bit and you have the real address.
3. On flash, use `hbreak`. A software breakpoint tries to modify read only memory and does not work.
4. A CRC is a fingerprint, not encryption. If you can recompute it, you can update it. Its strength depends on where it is stored.
5. A scrambled program is not hidden if the key is in the loader. XOR undoes itself, and the key schedule is a few instructions.
6. Code can run from RAM. Watch `pc` cross from `0x1000xxxx` to `0x2004xxxx` and you have seen it.
7. An off-by-one is a comparison that is one off. Find the exact instruction, then prove it by executing it.
8. A hidden command is usually present in the data and absent from the documentation. Dump the table and read the strings.
9. A challenge and response is beatable when the rule is reversible. Read the rule, compute the answer.
10. This lab cannot brick the board. The boot ROM survives everything, the shutdown path only writes RAM, and SWD can always halt the core.

Glossary

Term	Definition
`.bin`	raw bytes of the program, no names or map
`.elf`	program plus symbols and section map
SWD	two wire debug bus used by the probe
OpenOCD	program that speaks SWD and serves GDB
GDB	the debugger
Ghidra	static analysis and decompiler tool
`pc`	program counter, address of the next instruction
`sp`	stack pointer, top of the stack
`lr`	link register, return address
Thumb	the compact 16 and 32 bit ARM instruction encoding, selected by an odd address

Term	Definition
`hbreak`	hardware breakpoint, required for flash
XIP	execute in place, running code directly from flash
CRC-32	a 32 bit checksum used as a fingerprint
XOR	a mixing operation that undoes itself
LFSR	a shift register that produces a pseudo random sequence
xorshift	a fast pseudo random generator built from shifts and XOR
off-by-one	a bounds error that permits exactly one extra item
manifest	the stored CRC and company tag at `0x1003f000`
dead drop	a hidden location used to pass information
OTP	one time programmable memory, the only true brick risk

Appendix A: GDB and OpenOCD Quick Reference

Command	Purpose
`~/openocd/src/openocd -s ~/openocd/tcl -f interface/cmsis-dap.cfg -f target/rp2350.cfg -c "adapter speed 1000"`	start the debug server
`target remote :3333`	attach GDB
`monitor reset halt`	reset and stop
`monitor halt`	stop
`monitor rp2350.dap.core0 configure -rtos none`	disable RTOS awareness
`x/Nxw ADDR`	read N words in hex
`x/Nxb ADDR`	read N bytes in hex
`x/Ni ADDR`	disassemble N instructions
`hbreak *ADDR`	hardware breakpoint
`continue`	run to breakpoint
`si` / `ni`	step instruction, into or over
`finish`	run to function return
`info registers`	show registers
`set $rN = VALUE`	write a register

Command	Purpose
`set {int}ADDR = VALUE`	write four bytes
`set {char[N]}ADDR = "text"`	write a string
`printf "0x%08x\n", $r0`	print a register
`detach`	release the target

Appendix B: Address Map and Symbols

Real symbols from the shipped `.elf`, in address order. Use these to check your Ghidra names.

Address	Symbol	Meaning
`0x10000000`	`_vectors`	loader vector table
`0x10000008`	`Stage2_CRC32`	fingerprint routine
`0x10000054`	`Stage2_Xor`	scramble and unscramble
`0x10000088`	`Stage2_Reset`	loader entry
`0x100000fa`	`Stage2_Brick`	shutdown path
`0x10000130`	`Init_Stack`	set up stack
`0x10000150`	`Enable_Coprocessor`	enable coprocessor
`0x10001000`	`_app_vectors`	app vector table
`0x10001008`	`App_Reset`	app entry
`0x10001074`	`Console_PrintStr`	print a string
`0x10001090`	`Console_ReadLine`	read a line, has the bug
`0x100010c6`	`Console_Strcmp`	compare strings
`0x100010e8`	`Console_Dispatch`	walk the table
`0x10001118`	`Console_Loop`	banner, prompt, read, dispatch
`0x10001144`	`Cmd_Help`	help handler
`0x10001154`	`Cmd_Status`	status handler
`0x10001164`	`Cmd_Led`	led handler
`0x1000117c`	`Cmd_Unlock`	unlock handler
`0x1000118c`	`Cmd_Factory`	hidden factory handler
`0x10001620`	`msg_banner`	banner string
`0x1000168b`	`msg_help`	help text, no factory
`0x100016e1`	`msg_factory`	factory output

Address	Symbol	Meaning
`0x10001726`	`cmd_help`	name string
`0x10001748`	`cmd_table`	command table
`0x1003e000`	ghost blob	scrambled ghost in flash
`0x1003f000`	manifest	stored CRC and tag
`0x20000200`	`cmd_buf`	command buffer, 32 bytes
`0x20000220`	`cmd_handler`	adjacent word, overflow target
`0x20040000`	`Ghost_Entry`	ghost entry in RAM
`0x20040028`	`Ghost_Step`	xorshift step
`0x20040046`	`Ghost_Challenge`	issue a challenge
`0x20040058`	`Ghost_Check`	verify an answer
`0x200400bc`	`g_lfsr`	generator state
`0x200400c0`	`g_challenge`	current challenge

Appendix C: File Offsets

The image is loaded at `0x10000000`. To convert a file offset below to a memory address, add `0x10000000`. To go the other way, subtract `0x10000000`.

File offset	Memory address	Region
`0x00000`	`0x10000000`	loader vectors and code
`0x01000`	`0x10001000`	app vectors and code
`0x03e000`	`0x1003e000`	scrambled ghost
`0x03f000`	`0x1003f000`	manifest
`0x040000`	`0x10040000`	end of the 256 KB image

When you use `strings -t x`, the left column is a file offset. Inside the app, subtract `0x1000` from the file offset to get the offset within the app, then add `0x10001000` to get the address. Equivalently, for any offset in the first 256 KB, add `0x10000000` to get the loaded address.

End of Week 12

You took apart a deliberately defensive firmware, proved every claim with the real device, and can explain all of it. That is the capstone.